UNIVERSITATEA „DUNĂREA DE JOS” DIN GALAȚI
FACULTATEA DE AUTOMATICĂ, CALCULATOARE, INGINERIE ELECTRICĂ ȘI ELECTRONICĂ

Mini-Stație meteo

Motto: ”Vremea este imprevizibilă,
dar datele noastre nu”.

Îndrumător proiect/Coordonator științific,
Șef de lucrări doctor inginer Culea Mihai

Echipa: Robu Florin Octavian și Colția Petre Cristian
Master ”Tehnologii Informatice Avansate”
anul I, 2024 - 2025

Cuprins

Capitolul 1: Introducere

1.1 Necesitatea unei mini stații meteo

1.2 Caracterul educațional al proiectului

1.3 Obiective

Capitolul 2: Materiale utilizate

2.1. Materiale utilizate

2.2 Motivul alegerii acestora

Capitolul 3: Schema electrică

Capitolul 4: Calcul energetic

Capitolul 5: Software utilizat

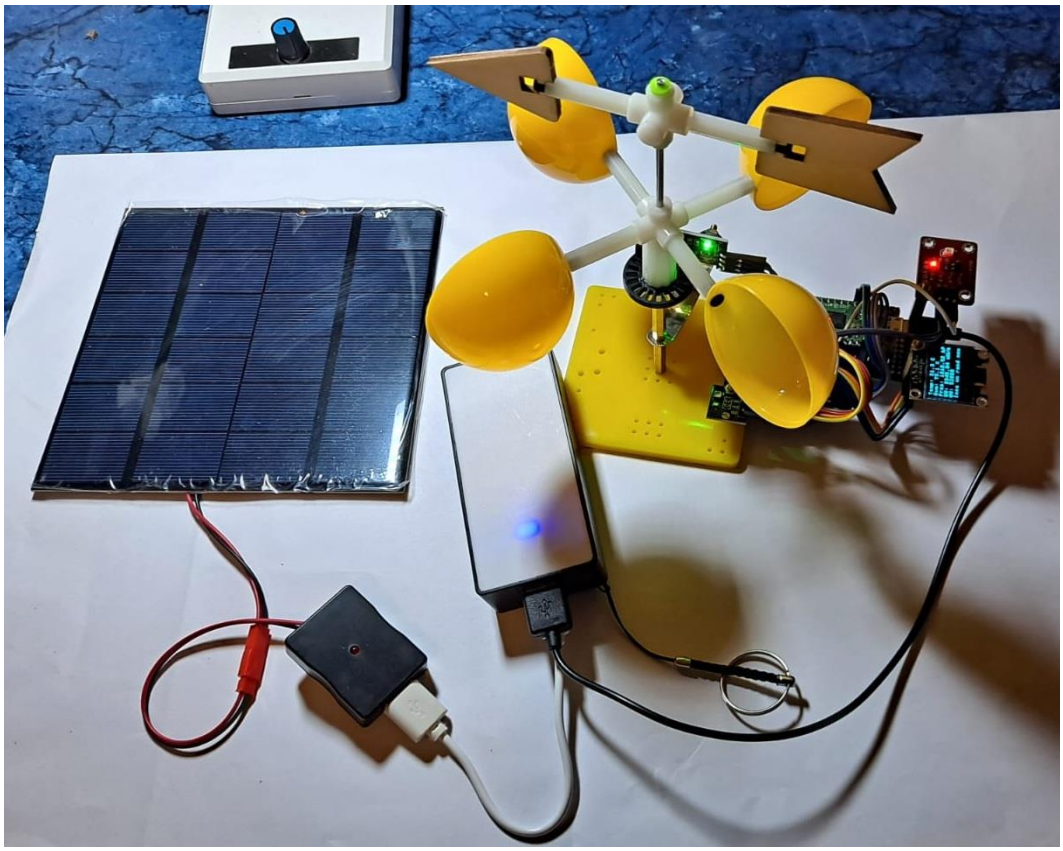
5.1. Programul MicroPython pentru Raspberry Pi Pico W

5.2. Fișierele necesare pentru server

5.3. Exemple de afișare pe pagina web

Capitolul 6: Calibrare senzori

Concluzii



Capitolul 1: Introducere

1.1 Necesitatea unei mini stații meteo

Monitorizarea condițiilor meteorologice este esențială într-o varietate de domenii, de la agricultură și meteorologie profesională, până la activități recreative și confortul personal. O mini-stație meteo, precum cea propusă, oferă posibilitatea de a obține date meteorologice precise și actualizate în timp real, direct de la locația dorită.

1.2 Caracterul educațional al proiectului

Construirea unei mini-stații meteo reprezintă o oportunitate excelentă de a aprofunda cunoștințele tehnice și de a dobândi experiență practică în domeniul IoT (Internet of Things) și al cloud computing-ului.

Proiectul implică:

• Lucrul cu microcontrolere (Raspberry Pi Pico W): programarea și configurarea microcontrolerului pentru a colecta și procesa date de la senzori;

• Interfațarea cu senzori: înțelegerea modului în care funcționează senzorii precum și a modului de a-i conecta la microcontroler;

• Programare în MicroPython: dezvoltarea de cod pentru a citi datele de la senzori, a le prelucra și a le transmite către cloud;

• Comunicare IoT și cloud: configurarea conexiunii Wi-Fi, transmiterea datelor către o bază de date cloud și vizualizarea datelor online;

• Înțelegerea bazelor de date cloud, și a metodelor de transfer a datelor către acestea.

1.3 Obiective

Mini stația meteo propusă va măsura următoarele valori fizice:

• temperatura;

• presiunea atmosferică;

• umiditatea atmosferică;

• radiațiile ultraviolete;

• viteza vântului;

Datele vor fi colectate și prelucrate în vederea transmiterii cu microcontrolerul Raspberry Pi Pico W (care dispune de un modul incorporat de comunicații WiFi)

Datele vor fi transmise către https://meteo.aswic.ro care beneficiază de protecție prin criptare, datorită utilizării protocolului HTTPS.

Această criptare asigură că informațiile sunt transformate într-un format ilizibil în timpul transferului, prevenind accesul neautorizat. În plus, prezența unui certificat digital valid asociat cu adresa web confirmă autenticitatea serverului, garantând că datele sunt trimise către o destinație legitimă și securizată.

Prin urmare, utilizarea HTTPS cu un certificat valid oferă un nivel important de securitate și confidențialitate pentru datele colectate și stocate pe https://meteo.aswic.ro.

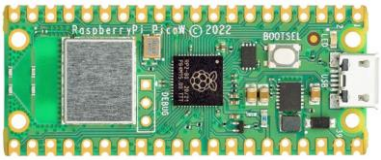
În paralel, datele vor fi colectate prin intermediul unui server XAMPP instalat pe un laptop, cu care vor fi realizate unele teste.

Totodată, ar mai putea fi utilizate servicii cloud specifice: platforme precum Adafruit IO, ThingSpeak sau Firebase, care oferă servicii de stocare și vizualizare a datelor IoT.

Capitolul 2: Materiale utilizate

2.1. Materiale utilizate

Raspberry Pi Pico W



Display-ul OLED (0.96 inch) SSD1306 (I2C)



Senzorul AHT20+BMP280 (temperatură, umiditate și presiune)



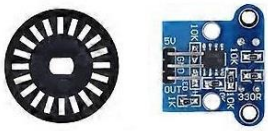
Senzorul GUVA-S12SD (nivel de radiații ultraviolete)



Dispozitiv pentru măsurarea vitezei vântului



Senzor fotoelectric cu impulsuri, pentru măsurarea vitezei de rotație a cupelor



Acumulatori pentru alimentare



Panou solar 3W, 65V – 500 mA



Regulator de 5V 2A



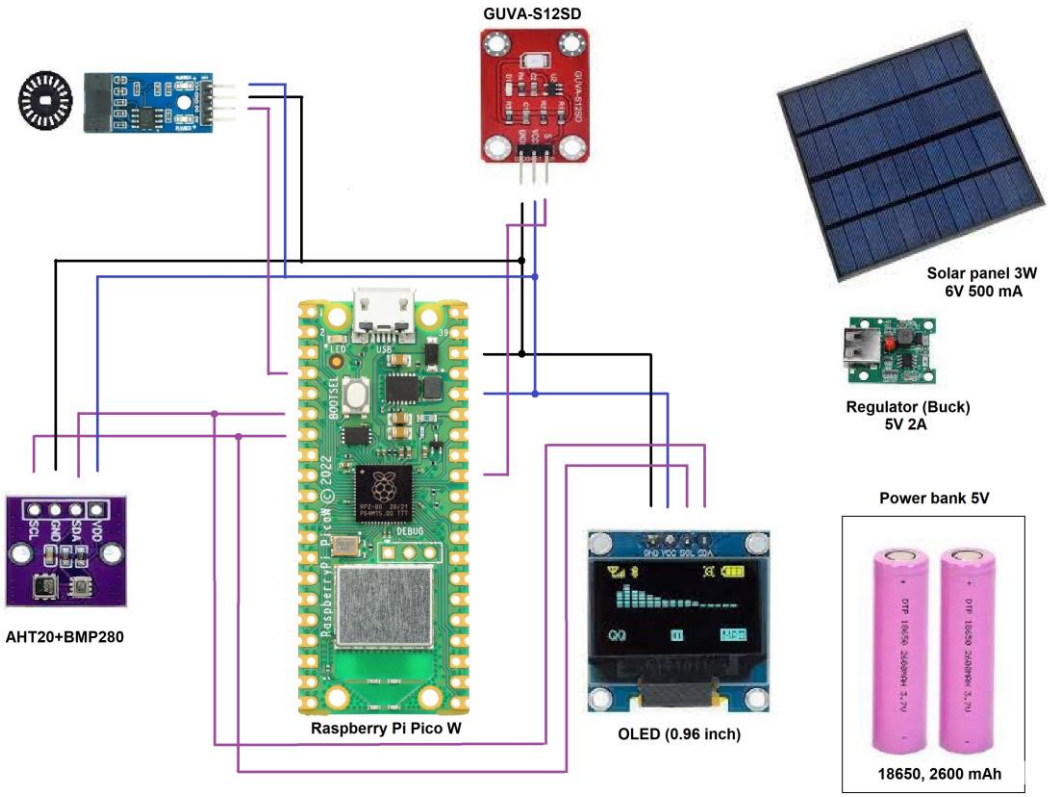
Fire cu terminații Dupont, șuruburi și altele.

2.2 Motivul alegerii acestor materiale

A fost ales Raspberry Pi Pico W pentru raportul preț/performanță și conectivitatea Wi-Fi integrată, display-ul OLED (0.96 inch) SSD1306 pentru consumul mic, iar senzorul AHT20+BMP280 pentru faptul că integrează într-un singur modul trei senzori, respectiv pentru temperatură, umiditate și presiune.

Dispozitivul pentru măsurarea vitezei vântului a permis instalarea unui senzor fotoelectric de măsurare a vitezei de rotație a cupelor. Acesta este un dispozitiv de jucărie, fapt pentru care nu a mai putut fi instalat un senzor magnetic de direcție pentru stabilirea direcției vântului. Girueta se mișca necontrolat, având doar caracter educativ.

Capitolul 3: Schema electrică



Conexiuni la Raspberry Pi Pico W:

Conectare pentru alimentarea senzorilor:

VCC - pin 36 (3.3V)

GND - pin 38 (GND)

Display-ul OLED (0.96 inch):

SDA - pin 6 (GPIO4)

SCL - pin 7 (GPIO5)

Senzorul AHT20+BMP280:

SDA - pin 6 (GPIO4)

SCL - pin 7 (GPIO5)

Senzorul GUVA-S12SD:

OUT – pin 32 (analogic)

Senzorul fotoelectric numărător de impulsuri

D0 – pin 4 (GPIO 2)

Capitolul 4: Calcul energetic

Consum microcontroller, display și senzori:

• Raspberry Pi Pico W: consum tipic între 17 mA și 27 mA la 3.3V, în funcție de modul de operare și de activitatea Wi-Fi. Consumul maxim poate ajunge la 300 mA în timpul transmisiilor Wi-Fi.

• OLED 0.96 inch: consum tipic între 15 mA și 30 mA la 3.3V, în funcție de luminozitatea afișajului și de numărul de pixeli aprinși.

• Senzorul AHT20 + BMP280:

AHT20: consum tipic de 1 mA la 3.3V.

BMP280: consum tipic de 0.3 mA la 3.3V.

• Senzorul GUVA-S12SD: consum tipic foarte scăzut, neglijabil, de ordinul microamperilor (µA).

• Senzorul fotoelectric: consum neglijabil, de ordinul microamperilor (µA) sau de câțiva miliamperi în timpul impulsurilor, fiind neglijabil în calculul energetic total al stației meteo.

Consum total: 27 mA (Pico W) + 30 mA (OLED) + 1 mA (AHT20) + 0.3 mA (BMP280) + consum neglijabil (GUVA-S12SD) + consum neglijabil (senzor fotoelectric) = aproximativ 58.3 mA la 3.3V.

Puterea consumată este = 0.0583 A x 3.3 V = aproximativ **0.192 wați**.

Alimentare: Stația va fi alimentată de doua baterii de 3.7 V tip 18650, 2600 mAh, dispuse în paralel într-un power bank, cu ieșire la 5 V. Capacitatea totală a bateriilor este de 5200 mAh.

Presupunând o eficiență de 80% (o valoare tipică) a convertorului DC-DC, Puterea de intrare este = Putere de ieșire / Eficiență = 0.192 W / 0.8 = 0.24 W.

Curentul de intrare la 5V = Putere de intrare / Tensiune de intrare = 0.24 W / 5 V = 0.048 A = 48 mA.

Timp de funcționare = Capacitate baterii (mAh) / Curent de intrare (mA) = 5200 mAh / 48 mA ≈ **108 ore**.

Deasemenea, mai putem adăuga un procent de 10-15 % pierderi, datorită cablurilor, conectorilor și a altor factori.

Sursă de alimentare regenerabilă: Un panou solar de 6V, 500mA va furniza energie unui regulator (Buck) de 5V 2A, asigurând o tensiune stabilă pentru power bank.

Power bank-ul va primi această tensiune și o va utiliza pentru a încărca bateriile interne și pentru a alimenta schema. Circuitul de încărcare al power bank-ului va gestiona automat încărcarea bateriilor, protejându-le de supraîncărcare și de alte probleme.

Presupunând o eficiență de 80% pentru regulatorul buck, puterea disponibilă la 5V este de 3W x 80% = 2.4W. Curentul disponibil la 5V este 2.4W / 5V = 0.48A = 480mA.

Curentul produs de panoul solar (480mA) este mai mare decât consumul schemei (48mA).

În condiții ideale, panoul solar poate compensa complet consumul schemei și poate încărca power bank-ul cu 432mA. Timpul necesar pentru a încărca complet power bank-ul de la 0% este: 5200mAh / 432mA = aproximativ 12 ore.

Dar, producția panoului solar depinde de intensitatea luminii solare. În zilele înnorate sau în timpul nopții, panoul solar va produce mai puțină energie sau deloc.

Concluzie: În condiții ideale, panoul solar poate prelungi semnificativ timpul de funcționare al schemei, sau chiar sa faca schema sa functioneze indefinit. În condiții reale, timpul de funcționare va varia în funcție de disponibilitatea luminii solare și de eficiența componentelor.

Capitolul 5: Software utilizat

Raspberry Pi Pico W va citi datele de la senzori și le va transmite către un server, în format Json.

Serverul va capta datele, le va salva intr-un fișier în format CSV, respectiv date_meteo.csv, respectiv le va afișa pe o pagina web. Acest lucru va fi realizat de două programe specializate aflate pe server, index.php și pico.php.

A fost aleasă această modalitate pentru a testa funcționalitatea dispozitivului, urmând ca în viitor datele să fie transmise în cloud.

5.1. Programul MicroPython care va fi încarcat în Raspberry Pi Pico W:

```
# proiect.py
from machine import Pin, ADC, I2C
import ssd1306
import bmp280
import ahtx0
import time
import network
import urequests
import json

# Configurarea I2C
i2c = I2C(0, scl=Pin(5), sda=Pin(4))

# Configurarea display-ului OLED
oled = ssd1306.SSD1306_I2C(128, 64, i2c)

# Configurarea senzorului BMP280 (adresa corectă: 0x77)
bmp = bmp280.BMP280(i2c, addr=0x77)

# Configurarea senzorului AHT20
aht = ahtx0.AHT20(i2c)

# Configurarea senzorului GUVA-S12SD
uv_sensor = ADC(27) # GPIO26 (Pin 32)
# Parametrii senzorului GUVA-S12SD
responsivitate = 0.14  # A/W
# curent_luminos_referinta = 112.5e-9 = 112.5 nA pentru 1 mW/cm² (media dintre 101 si 125)
curent_luminos_referinta = 101e-9  # limita inferioară
curent_intunecat = 1e-9  # 1 nA
bias_uv = 256 # valoarea bias
```

```
# Configurarea senzorului de vant (senzor de rotatie impulsuri)
wind_sensor_pin = Pin(2, Pin.IN, Pin.PULL_UP) # Pin 4 (GPIO2) ca intrare cu pull-up intern
impulsuri = 0
timp_anterior_vant = 0
viteza_vant = 0

def gestioneaza_impuls(pin):
    global impulsuri
    impulsuri += 1

wind_sensor_pin.irq(trigger=Pin.IRQ_FALLING, handler=gestioneaza_impuls)

wifi_config = [
    {"ssid": "NUME", "password": "PAROLA"}, # retea principala
    {"ssid": "NUME", "password": "PAROLA"} # retea secundara
]

wlan = network.WLAN(network.STA_IF)
wlan.active(True)

def connect_wifi(ssid, password):
    print(f"Încerc conectarea la {ssid}...")
    wlan.connect(ssid, password)
    for i in range(10):
        if wlan.isconnected():
            print(f"Conectat cu succes la {ssid}!")
            return True
        time.sleep(1)
    print(f"Nu s-a putut conecta la {ssid}.")
    return False

conectat = False
for config in wifi_config:
    if connect_wifi(config["ssid"], config["password"]):
        conectat = True
        break

if not conectat:
    print("Nu s-a putut conecta la nicio rețea Wi-Fi!")
else:
    print("Conectat la Wi-Fi")

url = "http://192.168.0.108/meteo/pico.php"  # Se inlocuieste cu adresa IP a site-ului web
headers = {'Content-Type': 'application/json; charset=utf-8'} # Specificam codificarea UTF-8

while True:
    try:
        # Citirea datelor de la BMP280
        temp_bmp = bmp.temperature
        pressure = bmp.pressure / 100  # Convertim Pa în hPa
```

```
# Citirea datelor de la AHT20
hum = aht.relative_humidity

# Citirea valorii analogice de la senzorul UV
uv_valoare_analogica = uv_sensor.read_u16()
uv_valoare_analogica_compensata = max(0, uv_valoare_analogica - bias_uv)
tensiune_uv = uv_valoare_analogica_compensata * 3.3 / 65535
curent_uv = tensiune_uv / 10000 # 10k ohmi, probabil
curent_uv_compensat = max(0, curent_uv - curent_intunecat)
intensitate_uv = curent_uv_compensat / curent_luminos_referinta

# Citirea și calcularea vitezei vântului
timp_curent_vant = time.ticks_ms()
delta_timp_vant = time.ticks_diff(timp_curent_vant, timp_anterior_vant)

if delta_timp_vant >= 1000:
    frecventa_vant = impulsuri / (delta_timp_vant / 1000) # impulsuri pe secundă
    factor_calibrare_vant = 0.032 # experimental
    viteza_vant = frecventa_vant * factor_calibrare_vant
    viteza_vant = viteza_vant * 3.6 # Km/h
    impulsuri = 0
    timp_anterior_vant = timp_curent_vant

# Formatare date cu două zecimale
temp_bmp_formatat = "{:.2f}".format(temp_bmp)
hum_formatat = "{:.2f}".format(hum)
pressure_formatat = "{:.2f}".format(pressure)
intensitate_uv_formatat = "{:.2f}".format(intensitate_uv)
wind_speed_formatat = "{:.2f}".format(viteza_vant)

# Formatați datele în JSON
date_meteo = {
    "temperatura": temp_bmp_formatat,
    "umiditate": hum_formatat,
    "presiune": pressure_formatat,
    "intensitate_uv": intensitate_uv_formatat,
    "viteza_vant": wind_speed_formatat
}

# Trimiteți datele JSON prin HTTP POST
try:
    date_json = json.dumps(date_meteo)
    #print("Date JSON trimise:", date_json) # Afiseaza datele JSON inainte de trimitere
    response = urequests.post(url, headers=headers, json=date_meteo)
    #print("Cod de stare HTTP:", response.status_code) # Afiseaza codul de stare HTTP
    print("Răspuns server:", response.text)
    #print("Date trimise cu succes:", response.text)
    response.close()
except OSError as e:
```

```
            print("Eroare la trimiterea datelor:", e)

        # Afișarea datelor pe display-ul OLED
        oled.fill(0)
        oled.text("Temp: {:.1f} C".format(temp_bmp), 0, 0) # Temp bmp.
        #oled.text("T2: {:.1f} C".format(temp_aht), 0, 10) # Temp aht.
        oled.text("Hum: {:.1f} %".format(hum), 0, 10)
        oled.text("Pres: {:.2f} hPa".format(pressure), 0, 20)
        oled.text("UV: {:.2f} mW/cm2".format(intensitate_uv), 0, 30)
        oled.text("Vant: {} Km/h".format(wind_speed_formatat), 0, 40)
        oled.text("O zi minunata", 0, 50)
        oled.show()

        # Afișarea datelor în terminal
        print("Temperatura: {:.1f} C".format(temp_bmp))
        #print("Temperatura AHT20: {:.1f} C".format(temp_aht))
        print("Umiditate: {:.1f} %".format(hum))
        print("Presiune: {:.2f} hPa".format(pressure))
        print("Intensitate UV: {:.2f} mW/cm2".format(intensitate_uv))
        print("UV: {}".format(uv_valoare_analogica))
        print("Viteza vant: {} Km/h".format(wind_speed_formatat))
        print("-" * 20)  # Linie de separator pentru claritate

    except OSError as e:
        print("Eroare la citirea senzorului")

    time.sleep(2)
```

5.2. Fișierele necesare pentru server

index.php

// afișează datele din baza de date csv

```
<?php
// Numele fișierului CSV
$nume_fisier_csv = 'date_meteo.csv';

// Numărul de valori implicit pentru grafic
$numar_valori_grafic_implicit = 30;

// Verifică dacă s-a trimis un număr de înregistrări prin GET
if (isset($_GET['nr_inregistrari']) && is_numeric($_GET['nr_inregistrari'])) {
    $numar_valori_grafic = intval($_GET['nr_inregistrari']);
    // Asigură-te că valoarea este în intervalul dorit (opțional)
    $valori_permise = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100];
    if (!in_array($numar_valori_grafic, $valori_permise)) {
        $numar_valori_grafic = $numar_valori_grafic_implicit; // Folosește valoarea implicită dacă nu este validă
    }
} else {
    $numar_valori_grafic = $numar_valori_grafic_implicit; // Folosește valoarea implicită dacă nu este setat
```

```
}

// Răspuns HTML pentru cererile GET
header('Content-Type: text/html; charset=utf-8');

// Citeste datele din fisierul csv
$date_csv = @file_get_contents($nume_fisier_csv);

if ($date_csv === false) {
    echo "<h1 class='titlu-meteo'>Date Meteo</h1>";
    echo "<p>Date CSV incorecte sau indisponibile.</p>";
} else {
    // Parsează datele CSV
    $linii = explode("\n", $date_csv);
    $date_meteo = [];
    foreach ($linii as $linie) {
        $valori = explode(",", $linie);
        if (count($valori) === 6) {
            $date_meteo[] = [
                'data_ora' => trim($valori[0]),
                'temperatura' => floatval(trim($valori[1])),
                'umiditate' => floatval(trim($valori[2])),
                'presiune' => floatval(trim($valori[3])),
                'intensitate_uv' => floatval(trim($valori[4])),
                'viteza_vant' => floatval(trim($valori[5])),
            ];
        }
    }

    // Inversează array-ul pentru a avea cele mai recente valori la început
    $date_meteo = array_reverse($date_meteo);

    // Afișează ultima înregistrare din fișierul CSV
    if (!empty($date_meteo)) {
        $ultima_inregistrare = $date_meteo[0];
        echo "<h1 class='titlu-meteo'>Ultima înregistrare meteo</h1>";
        echo "<table class='tabel-meteo'>";
        echo "<tr class='antet-tabel'>";
        echo "<th>Data/Ora</th>";
        echo "<th>Temperatură (°C)</th>";
        echo "<th>Umiditate (%)</th>";
        echo "<th>Presiune (hPa)</th>";
        echo "<th>Intensitate UV (mW/cm2)</th>";
        echo "<th>Viteza vantului (Km/h)</th>";
        echo "</tr>";
        echo "<tr>";
        echo "<td>" . $ultima_inregistrare['data_ora'] . "</td>";
        echo "<td>" . $ultima_inregistrare['temperatura'] . "</td>";
        echo "<td>" . $ultima_inregistrare['umiditate'] . "</td>";
        echo "<td>" . $ultima_inregistrare['presiune'] . "</td>";
```

```
    echo "<td>" . $ultima_inregistrare['intensitate_uv'] . "</td>";
    echo "<td>" . $ultima_inregistrare['viteza_vant'] . "</td>";
    echo "</tr>";
    echo "</table>";
} else {
    echo "<h1 class='titlu-meteo'>Date Meteo</h1>";
    echo "<p>Nu există înregistrări în fișierul CSV.</p>";
}

// Extrage ultimele n valori pentru grafice
$ultimele_valori = array_slice($date_meteo, 0, $numar_valori_grafic);

// Inversează din nou pentru a avea ordinea cronologică corectă pentru grafice
$ultimele_valori = array_reverse($ultimele_valori);

// Prepare data for charts
$labels = array_column($ultimele_valori, 'data_ora');
$temperaturi = array_column($ultimele_valori, 'temperatura');
$umiditati = array_column($ultimele_valori, 'umiditate');
$presiuni = array_column($ultimele_valori, 'presiune');
$uv_intensitati = array_column($ultimele_valori, 'intensitate_uv');
$viteze_vant = array_column($ultimele_valori, 'viteza_vant');

echo "<h2 class='titlu-grafice'>Grafice meteo, pentru ultimile ";
echo "<select onchange='schimbaNrInregistrari(this)'>";
$valori_combo = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100];
foreach ($valori_combo as $valoare) {
    $selected = ($valoare == $numar_valori_grafic) ? 'selected' : '';
    echo "<option value='" . $valoare . "' " . $selected . ">" . $valoare . "</option>";
}
echo "</select>";
echo " înregistrări</h2>";
echo "<div class='container-grafice'>";

// Grafic Temperatură
echo "<div class='grafic'>";
echo "<h3>Temperatură (°C)</h3>";
echo "<canvas id='graficTemperatura'></canvas>";
echo "</div>";

// Grafic Umiditate
echo "<div class='grafic'>";
echo "<h3>Umiditate (%)</h3>";
echo "<canvas id='graficUmiditate'></canvas>";
echo "</div>";

// Grafic Presiune
echo "<div class='grafic'>";
echo "<h3>Presiune (hPa)</h3>";
echo "<canvas id='graficPresiune'></canvas>";
```

```
    echo "</div>";

    // Grafic Intensitate UV
    echo "<div class='grafic'>";
    echo "<h3>Intensitate UV (mW/cm2)</h3>";
    echo "<canvas id='graficUV'></canvas>";
    echo "</div>";

    // Grafic Viteza Vântului
    echo "<div class='grafic'>";
    echo "<h3>Viteza Vântului (Km/h)</h3>";
    echo "<canvas id='graficVant'></canvas>";
    echo "</div>";

    echo "</div>";
}
?>

<link rel="stylesheet" href="style.css">
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<script>
    const labelsTemperatura = <?php echo json_encode($labels); ?>;
    const dataTemperatura = {
        labels: labelsTemperatura,
        datasets: [{
            label: 'Temperatură (°C)',
            data: <?php echo json_encode($temperaturi); ?>,
            borderColor: 'rgb(255, 99, 132)',
            backgroundColor: 'rgba(255, 99, 132, 0.2)',
            tension: 0.1
        }]
    };
    const configTemperatura = {
        type: 'line',
        data: dataTemperatura,
        options: {
            responsive: true,
            scales: {
                y: {
                    beginAtZero: false
                }
            }
        }
    };
    const graficTemperatura = new Chart(
        document.getElementById('graficTemperatura'),
        configTemperatura
    );

    const labelsUmiditate = <?php echo json_encode($labels); ?>;
```

```
const dataUmiditate = {
    labels: labelsUmiditate,
    datasets: [{
        label: 'Umiditate (%)',
        data: <?php echo json_encode($umiditati); ?>,
        borderColor: 'rgb(54, 162, 235)',
        backgroundColor: 'rgba(54, 162, 235, 0.2)',
        tension: 0.1
    }]
};
const configUmiditate = {
    type: 'line',
    data: dataUmiditate,
    options: {
        responsive: true,
        scales: {
            y: {
                beginAtZero: false
            }
        }
    }
};
const graficUmiditate = new Chart(
    document.getElementById('graficUmiditate'),
    configUmiditate
);

const labelsPresiune = <?php echo json_encode($labels); ?>;
const dataPresiune = {
    labels: labelsPresiune,
    datasets: [{
        label: 'Presiune (hPa)',
        data: <?php echo json_encode($presiuni); ?>,
        borderColor: 'rgb(75, 192, 192)',
        backgroundColor: 'rgba(75, 192, 192, 0.2)',
        tension: 0.1
    }]
};
const configPresiune = {
    type: 'line',
    data: dataPresiune,
    options: {
        responsive: true,
        scales: {
            y: {
                beginAtZero: false
            }
        }
    }
};
```

```
const graficPresiune = new Chart(
    document.getElementById('graficPresiune'),
    configPresiune
);

const labelsUV = <?php echo json_encode($labels); ?>;
const dataUV = {
    labels: labelsUV,
    datasets: [{
        label: 'Intensitate UV (mW/cm2)',
        data: <?php echo json_encode($uv_intensitati); ?>,
        borderColor: 'rgb(255, 205, 86)',
        backgroundColor: 'rgba(255, 205, 86, 0.2)',
        tension: 0.1
    }]
};
const configUV = {
    type: 'line',
    data: dataUV,
    options: {
        responsive: true,
        scales: {
            y: {
                beginAtZero: false
            }
        }
    }
};
const graficUV = new Chart(
    document.getElementById('graficUV'),
    configUV
);

const labelsVant = <?php echo json_encode($labels); ?>;
const dataVant = {
    labels: labelsVant,
    datasets: [{
        label: 'Viteza Vântului (Km/h)',
        data: <?php echo json_encode($viteze_vant); ?>,
        borderColor: 'rgb(153, 102, 255)',
        backgroundColor: 'rgba(153, 102, 255, 0.2)',
        tension: 0.1
    }]
};
const configVant = {
    type: 'line',
    data: dataVant,
    options: {
        responsive: true,
        scales: {
```

```
            y: {
                beginAtZero: false
            }
        }
    }
};
const graficVant = new Chart(
    document.getElementById('graficVant'),
    configVant
);

function schimbaNrInregistrari(selectObject) {
    let selectedValue = selectObject.value;
    window.location.href = window.location.pathname + '?nr_inregistrari=' + selectedValue;
}

// Reîncarcă pagina la fiecare 5 secunde
setInterval(function() {
    location.reload();
}, 5000);
</script>
```

pico.php

//primește și salvează datele de la microcontroller

```
<?php
// Numele fișierului CSV
$nume_fisier_csv = 'date_meteo.csv';

// Verifică dacă există date JSON în cererea POST
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_SERVER['CONTENT_TYPE']) && strpos($_SERVER['CONTENT_TYPE'], 'application/json') !== false) {
    // Obține datele JSON din cerere
    $date_json = file_get_contents('php://input');
    $date_meteo = json_decode($date_json, true);

    // Verifică dacă datele JSON sunt valide
    if ($date_meteo && is_array($date_meteo) && isset($date_meteo['temperatura']) && isset($date_meteo['umiditate']) && isset($date_meteo['presiune']) && isset($date_meteo['intensitate_uv']) && isset($date_meteo['viteza_vant'])) {
        // Adaugă data și ora curente
        $data_ora = date('Y-m-d H:i:s');

        // Formatează datele pentru fișierul CSV
        $linie_csv = implode(',', [
            $data_ora,
            $date_meteo['temperatura'],
            $date_meteo['umiditate'],
            $date_meteo['presiune'],
            $date_meteo['intensitate_uv'],
```

```
                                    $date_meteo['viteza_vant'],
        ]) . "\n";

        // Scrie datele în fișierul CSV
        file_put_contents($nume_fisier_csv, $linie_csv, FILE_APPEND);

        // Răspuns JSON către Pico W
        header('Content-Type: application/json');
        echo json_encode(['success' => true, 'message' => 'Date primite și procesate cu succes.']);
        exit;
    } else {
        // Răspuns JSON de eroare pentru Pico W
        header('Content-Type: application/json');
        echo json_encode(['success' => false, 'message' => 'Date JSON incorecte sau lipsesc.']);
        exit;
    }
} else {
    // Răspuns JSON de eroare pentru Pico W
    header('Content-Type: application/json');
    echo json_encode(['success' => false, 'message' => 'Cerere incorectă.']);
    exit;
}
?>
```

style.css

```
/* style.css */

.titlu-meteo {
    color: #4682B4;
}

.tabel-meteo {
    width: 100%;
    border-collapse: collapse;
    border: 1px solid #ddd;
}

.antet-tabel {
    background-color: #ADD8E6;
}

.tabel-meteo th, .tabel-meteo td {
    padding: 8px;
    text-align: left;
    border: 1px solid #ddd;
}

.tabel-meteo th {
```

```
    width: auto;
}

.container-grafice {
    display: flex; /* Folosim Flexbox pentru a aranja graficele pe orizontală */
    flex-wrap: wrap; /* Permitem graficelor să treacă pe rândul următor dacă nu încap */
    justify-content: space-around; /* Distribuie spațiul în jurul graficelor */
}

.grafic {
    width: calc(50% - 20px); /* Fiecare grafic va ocupa aproximativ jumătate din lățime, cu puțin spațiu între ele */
    margin-bottom: 20px; /* Adaugă puțin spațiu sub fiecare grafic */
    box-sizing: border-box; /* Include padding și border în lățimea și înălțimea elementului */
}

.grafic canvas {
    width: 100% !important; /* Asigură că lățimea canvas-ului se adaptează la container */
    height: auto !important; /* Menține aspectul proporțional al graficului */
}

/* Pentru ecrane mai mici, fă graficele să ocupe toată lățimea */
@media (max-width: 768px) {
    .grafic {
        width: 100%;
    }
}

select {
  padding: 8px 12px;
  border: 1px solid #ccc;
  border-radius: 4px;
  font-size: 14px;
  color: #333;
  background-color: #f8f8f8;
  appearance: none; /* Elimină aspectul implicit al browserului */
  -webkit-appearance: none; /* Pentru Safari */
  -moz-appearance: none; /* Pentru Firefox */
  background-image: url('data:image/svg+xml;utf8,<svg fill="%23333" height="24" viewBox="0 0 24 24" width="24" xmlns="http://www.w3.org/2000/svg"><path d="M7 10l5 5 5-5z"/><path d="M0 0h24v24H0z" fill="none"/></svg>'); /* Adaugă o săgeată personalizată */

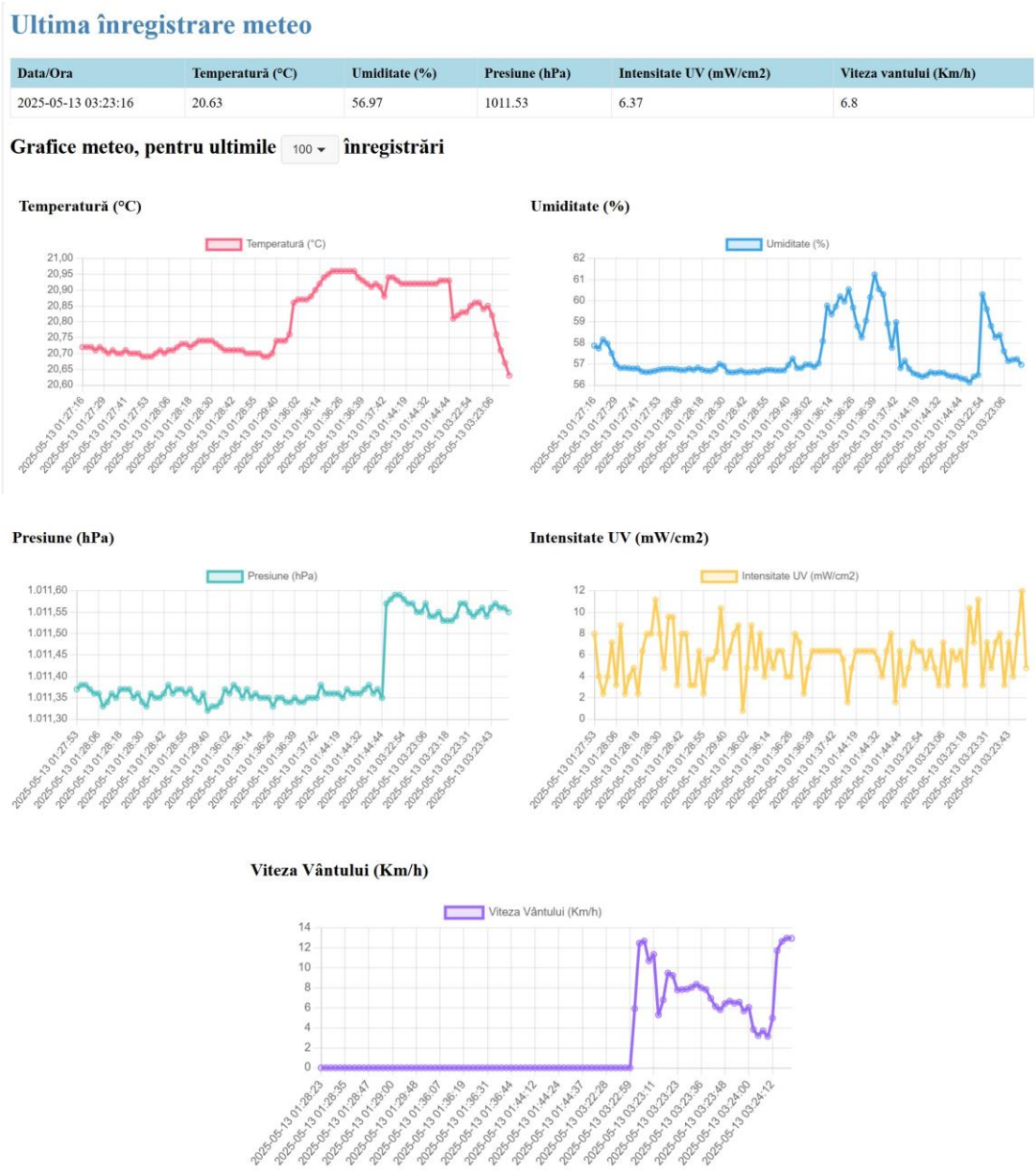
  background-repeat: no-repeat;
  background-position: right 8px top 50%;
  padding-right: 30px; /* Spațiu pentru săgeată */
}

select:focus {
  border-color: #66afe9;
  outline: none;
  box-shadow: 0 0 5px rgba(102, 175, 233, 0.5);
```

```
}

select option {
  padding: 8px 12px;
  font-size: 14px;
  color: #333;
}
```

5.3. Exemple de afișare pe pagina web



Capitolul 6: Calibrare senzori

• Senzorul AHT20 + BMP280 este deja calibrat din fabrică și afișeaza valori corecte pentru temperatură, umiditate și presiune.

• Senzorul GUVA-S12SD a necesitat calibrare, care a fost realizată in programul MicroPython.

Caracteristici GUVA-S12SD 3528 Ultraviolet Sensor:

Detecting Range of Spectrum: 240-370nm

Active Region：0.076 mm2

Responsivity: 0.14A/W

Dark Current: 1nA

Light Current:101~125nA UVA Light, 1mW/cm2

S-a calculat curentul prin senzor folosind tensiunea și o rezistență de 10k ohmi (utilizate frecvent în circuitele electronice pentru a limita curentul și a crea divizoare de tensiune). S-a compensat curentul întunecat. S-a calculat intensitatea UV în mW/cm² folosind curentul compensat și curentul luminos de referință.

Senzorul UV a fost testat cu un sterilizator UV cu led, cu o putere de 0.4 W și lungime de undă în intervalul 200 – 280 nm, spectru care se intersectează într-o anumită măsura cu senzorul nostru.

• Senzorul fotoelectric de impulsuri a fost calibrat cu ajutorul unui anemomentru si a unui ventilator. S-a stabilit că ventilatorul mișcă aerul cu o viteză de aproximativ 4 m/s, sau 14.4 Km/h, valoare care a stat la baza stabilirii unui factor de calibrare experimental în programul Python de 0.032.

Concluzii

Proiectul mini stației meteo a demonstrat cu succes capacitatea de a colecta și afișa în timp real date meteorologice esențiale, atingând obiectivele propuse.

Prin intermediul acestui proces practic, au fost consolidate cunoștințele despre programarea microcontrolerului Raspberry Pi Pico W în MicroPython, interfațarea cu o varietate de senzori meteorologici și principiile de bază ale comunicării IoT și ale transferului de date către un server web.

Deși performanța stației este influențată de factori precum precizia senzorilor și stabilitatea conexiunii Wi-Fi, prototipul oferă o platformă funcțională pentru monitorizarea locală a condițiilor meteo.

Direcțiile viitoare de dezvoltare includ explorarea stocării datelor în cloud prin platforme dedicate IoT, extinderea gamei de senzori monitorizați și optimizarea consumului energetic pentru o funcționare mai îndelungată pe baterii și energie solară.

În concluzie, acest proiect reprezintă o inițiativă valoroasă atât din punct de vedere educațional, oferind o experiență practică semnificativă în domeniul tehnologiilor emergente, cât și ca un prim pas spre crearea unei soluții personalizate de monitorizare meteo.

